Q5.1 — Implementing pes checkout <branch>

A branch in your system is simply a file inside .pes/refs/heads/ that stores a commit hash, so checking out a branch fundamentally means redirecting HEAD to point to that branch reference. The first step is to update .pes/HEAD so that it contains ref: refs/heads/<branch>. Next, you read the commit hash stored in that branch file and load the corresponding commit object from the object store, followed by its root tree. The working directory must then be updated to exactly match this tree: for every file in the target tree, you retrieve its blob content and write it to disk, and for any file that exists in the current working directory but not in the target tree, you remove it. After synchronizing the filesystem, you rebuild the index so that it reflects the state of the checked-out commit. The complexity arises because this is not just a pointer switch; it requires carefully reconciling differences between two snapshots while ensuring that no user changes are accidentally overwritten, which involves comparing states, handling deletions, and maintaining consistency across the working directory and index.

Q5.2 — Detecting a dirty working directory conflict

To detect whether a checkout should be refused due to uncommitted changes, you only need the index and the object store. For each tracked file, you first compare the working directory version with the index by recomputing the file's hash and checking if it matches the stored hash in the index; if it differs, the file has been modified locally. Next, you compare the index version of the file with the version in the target branch by retrieving the corresponding blob hash from the target commit's tree. If these hashes differ, it means the file has different content across branches. A conflict occurs when both conditions are true simultaneously: the user has modified the file locally, and the target branch also has a different version of that file. In this case, checkout must be refused to prevent overwriting the user's uncommitted work. This approach works efficiently because the index acts as the baseline snapshot, and the object store provides the target snapshot, eliminating the need for historical tracking.

Q5.3 — Detached HEAD behavior and recovery

A detached HEAD state occurs when HEAD directly contains a commit hash instead of pointing to a branch reference. In this state, if you make new commits, they are created normally but are not associated with any branch, meaning no reference points to them. As a result, these commits become effectively "dangling" and are at risk of being lost if you later switch branches, since no branch preserves their history. However, recovery is still possible. If the user realizes before switching, they can create a new branch pointing to the current commit, which preserves the work. Even after switching, recovery can be achieved by using a reflog-like mechanism or by manually identifying the commit hash and creating a new branch that points to it. The key idea is that commits themselves are not immediately

deleted; they are only at risk because they are not referenced, so creating a reference restores them to safety.

Q6.1 — Garbage collection algorithm and reachability

Garbage collection in this system can be implemented using a classic mark-and-sweep approach. The process begins by identifying all reachable objects starting from the branch heads stored in .pes/refs/heads/. Each branch head provides a commit hash, and from there, you recursively traverse the commit graph by following parent pointers and associated tree objects. For each tree, you further traverse all referenced blobs and subtrees. During this traversal, you maintain a hash set of visited object hashes to efficiently track reachability and avoid revisiting the same objects. After marking all reachable objects, you perform the sweep phase by iterating through all objects in .pes/objects/ and deleting those that are not present in the reachable set. For a repository with around 100,000 commits and 50 branches, the total number of unique commits is typically closer to 100,000 rather than 5 million due to shared history. Including trees and blobs, the traversal would likely involve several hundred thousand objects, making an efficient hash-based lookup essential for performance.

Q6.2 — GC race condition and safety mechanisms

Running garbage collection concurrently with a commit operation is dangerous because of potential race conditions. During a commit, new objects such as blobs, trees, and commits are created and written to the object store before the branch reference is updated to point to the new commit. If garbage collection runs at the same time, it may scan the repository, determine that these newly created objects are not reachable from any existing branch, and delete them before the commit operation completes. This leads to a situation where the branch reference ends up pointing to objects that no longer exist, effectively corrupting the repository. Real Git avoids this problem through a combination of strategies. It ensures that object creation happens before reference updates, making the process atomic from the perspective of reachability. It also uses locking mechanisms and carefully coordinated operations so that garbage collection does not interfere with active writes. Additionally, Git employs safety measures like grace periods and reference logs to prevent premature deletion of recently created objects, ensuring repository integrity even under concurrent operations.

```
yajat@Yajats-HP:/mnt/c/Users/yajat/OneDrive/Subjects/OS/OrangeProblemOS/os-u4-orange-problem$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
yajat@Yajats-HP:/mnt/c/Users/yajat/OneDrive/Subjects/OS/OrangeProblemOS/os-u4-orange-problem$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

```
yajat@Yajats-HP:/mnt/c/Users/yajat/OneDrive/Subjects/OS/OrangeProblemOS/os-u4-orange-problem$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

```
yajat@Yajats-HP:/mnt/c/Users/yajat/OneDrive/Subjects/OS/OrangeProblemOS/os-u4-orange-problem$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
/usr/bin/ld: tree.o: in function `tree_from_index':
tree.c:(.text+0x3cd): undefined reference to `index_load'
collect2: error: ld returned 1 exit status
make: *** [Makefile:22: test_tree] Error 1
yajat@Yajats-HP:/mnt/c/Users/yajat/OneDrive/Subjects/OS/OrangeProblemOS/os-u4-orange-problem$ ./test_tree
-bash: ./test_tree: No such file or directory
```

```
yajat@Yajats-HP:/mnt/c/Users/yajat/OneDrive/Subjects/OS/OrangeProblemOS/os-u4-orange-problem$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
yajat@Yajats-HP:/mnt/c/Users/yajat/OneDrive/Subjects/OS/OrangeProblemOS/os-u4-orange-problem$ xxd .pes/objects/XX/YYY... | head -20
xxd: .pes/objects/XX/YYY...: No such file or directory
yajat@Yajats-HP:/mnt/c/Users/yajat/OneDrive/Subjects/OS/OrangeProblemOS/os-u4-orange-problem$
Ln 176, Col 21   Spaces: 4   UTF-8   LF   { }
```

```
yajat@Yajats-HP:/mnt/c/Users/yajat/OneDrive/Subjects/OS/OrangeProblemOS/os-u4-orange-problem$ ./pes init
Initialized empty PES repository in .pes/
```

```
yajat@Yajats-HP:/mnt/c/Users/yajat/OneDrive/Subjects/OS/OrangeProblemOS/os-u4-orange-problem$ ./pes add file1.txt
yajat@Yajats-HP:/mnt/c/Users/yajat/OneDrive/Subjects/OS/OrangeProblemOS/os-u4-orange-problem$ ./pes status
Staged changes:
  staged:     file1.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.c
  untracked:  index.h
  untracked:  Makefile
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  README.md
  untracked:  test_objects
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h
```

```
yajat@Yajats-HP:/mnt/c/Users/yajat/OneDrive/Subjects/OS/OrangeProblemOS/os-u4-orange-problem$ cat .pes/index
100777 66224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39 1776508649 6 file1.txt
```

```
yajat@Yajats-HP:/mnt/c/Users/yajat/OneDrive/Subjects/OS/OrangeProblemOS/os-u4-orange-problem$ make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
yajat@Yajats-HP:/mnt/c/Users/yajat/OneDrive/Subjects/OS/OrangeProblemOS/os-u4-orange-problem$ make
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function ‘object_write’:
object.c:104:45: warning: ‘.tmp’ directive output may be truncated writing 4 bytes into a region of size between 1 and 512 [-Wformat-truncation=]
  104 |     snprintf(tmp_path, sizeof(tmp_path), "%s.tmp", path);
      |                                              ^~~~
In file included from /usr/include/stdio.h:980,
                 from pes.h:8,
                 from object.c:3:
In function ‘snprintf’,
    inlined from ‘object_write’ at object.c:104:5:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:54:10: note: ‘__builtin___snprintf_chk’ output between 5 and 516 bytes into a destination of size 512
   54 |   return __builtin___snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
      |          ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   55 |                                    __glibc_objsize (__s), __fmt,
      |                                    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   56 |                                    __va_arg_pack ());
      |                                    ~~~~~~~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
index.c: In function ‘index_add’:
index.c:183:41: warning: comparison of integer expressions of different signedness: ‘size_t’ {aka ‘long unsigned int’} and ‘__off_t’ {aka ‘long int’} [-Wsign-compare
]
  183 |         fread(buffer, 1, st.st_size, f) != st.st_size) {
      |                                         ^~
index.c:192:9: warning: implicit declaration of function ‘object_write’ [-Wimplicit-function-declaration]
  192 |     if (object_write(OBJ_BLOB, buffer, st.st_size, &hash) != 0) {
      |         ^~~~~~~~~~~
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
```